UNIT-1 Introduction to Python

What is Python

Python is a general-purpose, dynamically typed, high-level, compiled and interpreted, garbage collected, and purely object-oriented programming language that supports procedural, object oriented, and functional programming

Features of Python:

- **Easy to use and Read -** Python's syntax is clear and easy to read, making it an ideal language for both beginners and experienced programmers. This simplicity can lead to faster development and reduce the chances of errors.
- **Dynamically Typed** - The data types of variables are determined during run-time. We do not need to specify the data type of a variable during writing codes.
- **High-level** - High-level language means human readable code.
- **Compiled and Interpreted** - Python code first gets compiled into bytecode, and then interpreted line by line. When we download the Python in our system form org we download the default implement of Python known as CPython. CPython is considered to be Complied and Interpreted both.
- **Garbage Collected** - Memory allocation and de-allocation are automatically managed. Programmers do not specifically need to manage the memory.
- **Purely Object-Oriented** - It refers to everything as an object, including numbers and strings.
- **Cross-platform Compatibility** - Python can be easily installed on Windows, macOS, and various Linux distributions, allowing developers to create software that runs across different operating systems.
- **Rich Standard Library** - Python comes with several standard libraries that provide ready-to-use modules and functions for various tasks, ranging from **web development** and **data manipulation** to **machine learning** and **networking**.
- **Open Source** - Python is an open-source, cost-free programming language. It is utilized in several sectors and disciplines as a result.

Printing 'Hello World'

Python Code:

```
print("Hello World)"
```

Java Code:

```
public class HelloWorld {
 public static void main(String[] args) {
 System.out.println("Hello, World!");
 }
}
```

Python Basic Syntax

There is no use of curly braces or semicolons in Python programming language. It is an English like language. But Python uses indentation to define a block of code. Indentation is nothing but adding whitespace before the statement when it is needed.

For example -

```
def func():
 statement 1
 statement 2
 ……————————————
 ……————————————
 statement N
```

In the above example, the statements that are the same level to the right belong to the function. Generally, we can use four whitespaces to define indentation.

Instead of Semicolon as used in other languages, Python ends its statements with a NewLine character.

Python is a case-sensitive language, which means that uppercase and lowercase letters are treated differently. For example, 'name' and 'Name' are two different variables in Python.

In Python, comments can be added using the '#' symbol. Any text written after the '#' symbol is considered a comment and is ignored by the interpreter. This trick is useful for adding notes to the code or temporarily disabling a code block. It also helps in understanding the code better by some other developers.

'If', 'otherwise', 'for', 'while', 'try', 'except', and 'finally' are a few reserved keywords in Python

that cannot be used as variable names. These terms are used in the language for particular

reasons and have fixed meanings. If you use these keywords, your code may include errors, or the interpreter may reject them as potential new Variables.

History of Python

Python was created by Guido van Rossum. In the late 1980s, Guido van Rossum, a Dutch programmer, began working on Python while at the Centrum Wiskunde& Informatica (CWI) in the Netherlands. He wanted to create a successor to the **ABC programming language** that would be easy to read and efficient.

In February 1991, the first public version of Python, version 0.9.0, was released. This marked the official birth of **Python as an open-source project**. The language was named after the British comedy series "**Monty Python's Flying Circus**".

Python development has gone through several stages. **In January 1994, Python 1.0 was released as a usable and stable programming language.** This version included many of the features that are still present in Python today.

From the 1990s to the 2000s, Python gained popularity for its simplicity, readability, and versatility. **In October 2000, Python 2.0 was released**. Python 2.0 introduced list comprehensions, garbage collection, and support for Unicode.

In December 2008, Python 3.0 was released. Python 3.0 introduced several backward incompatible changes to improve code readability and maintainability.

Throughout 2010s, Python's popularity increased, particularly in fields like data science, machine learning, and web development. Its rich ecosystem of libraries and frameworks made it a favourite among developers.

The **Python Software Foundation (PSF) was established in 2001** to promote, protect, and advance the Python programming language and its community.

Why learn Python?

Python provides many useful features to the programmer. These features make it the most popular and widely used language. We have listed below few-essential features of Python.

- **Easy to use and Learn:** Python has a simple and easy-to-understand syntax, unlike traditional languages like C, C++, Java, etc., making it easy for beginners to learn.
- **Expressive Language:** It allows programmers to express complex concepts in just a few

lines of code or reduces Developer's Time.

- **Interpreted Language:** Python does not require compilation, allowing rapid development and testing. It uses Interpreter instead of Compiler.

- **Object-Oriented Language**: It supports object-oriented programming, making writing reusable and modular code easy.
- **Open-Source Language:** Python is open-source and free to use, distribute and modify. ○ **Extensible:** Python can be extended with modules written in C, C++, or other languages. ○ **Learn Standard Library:** Python's standard library contains many modules and functions that can be used for various tasks, such as string manipulation, web programming, and more.
- **GUI Programming Support:** Python provides several GUI frameworks, such as Tkinter and PyQt, allowing developers to create desktop applications easily. ○ **Integrated:** Python can easily integrate with other languages and technologies, such as C/C++, Java, and . NET.
- **Embeddable:** Python code can be embedded into other applications as a scripting language.
- **Dynamic Memory Allocation:** Python automatically manages memory allocation, making it easier for developers to write complex programs without worrying about memory management.
- **Wide Range of Libraries and Frameworks:** Python has a vast collection of libraries and frameworks, such as NumPy, Pandas, Django, and Flask, that can be used to solve a wide range of problems.
- **Versatility:** Python is a universal language in various domains such as web development, machine learning, data analysis, scientific computing, and more. ○ **Large Community:** Python has a vast and active community of developers contributing to its development and offering support. This makes it easy for beginners to get help and learn from experienced developers.
- **Career Opportunities:** Python is a highly popular language in the job market. Learning Python can open up several career opportunities in data science, artificial intelligence, web development, and more.
- **High Demand:** With the growing demand for automation and digital transformation, the need for Python developers is rising. Many industries seek skilled Python developers to

help build their digital infrastructure.

- **Increased Productivity:** Python has a simple syntax and powerful libraries that can help developers write code faster and more efficiently. This can increase productivity and save time for developers and organizations.
- **Big Data and Machine Learning:** Python has become the go-to language for big data and machine learning. Python has become popular among data scientists and machine learning engineers with libraries like NumPy, Pandas, Scikit-learn, TensorFlow, and more.

Python Popular Frameworks and Libraries

Python has wide range of libraries and frameworks widely used in various fields such as machine learning, artificial intelligence, web applications, etc. We define some popular frameworks and libraries of Python as follows.

- **Web development (Server-side) - Django Flask**, **Pyramid**, **CherryPy**
- **GUIs based applications - Tkinter**, **PyGTK**, **PyQt**, **PyJs**, etc.
- **Machine Learning - TensorFlow**, **PyTorch**, **Scikit-learn**, **Matplotlib**, **Scipy**, etc.
- **Mathematics - NumPy**, **Pandas**, etc.
- **BeautifulSoup:** a library for web scraping and parsing HTML and XML
- **Requests**: a library for making HTTP requests
- **SQLAlchemy**: a library for working with SQL databases
- **Kivy:** a framework for building multi-touch applications
- **Pygame:** a library for game development
- **Pytest:** a testing framework for Python Django
- **REST framework**: a toolkit for building RESTful APIs
- **FastAPI**: a modern, fast web framework for building APIs
- **Streamlit**: a library for building interactive web apps for machine learning and data science
- **NLTK**: a library for natural language processing

Writing our first program:

Just type in the following code after you start the interpreter.

Writing our first program:


```
# Script Begins

print("STEFFY")

# Scripts Ends
```

Output:

STEFFY

***Line 1: [*# Script Begins]** In Python, comments begin with a #. This statement is ignored by the interpreter and serves as documentation for our code.

Line 2: [print("STEFFY")] To print something on the console, print() function is used. This function also adds a newline after our message is printed(unlike in C). Note that in Python 2, "print" is not a function but a keyword and therefore can be used without parentheses. However, in Python 3, it is a function and must be invoked with parentheses.

***Line 3: [*# Script Ends]** This is just another comment like in Line 1.

Python designed by Guido van Rossum at CWI has become a widely used general-purpose, high level programming language.

Python vs JAVA

Python	**Java**
Dynamically Typed • No need to declare anything. An assignment statement binds a name to an object, and the object can be of any type. • No type casting is required when using container objects	**Statically Typed** • All variable names (along with their types) must be explicitly declared. Attempting to assign an object of the wrong type to a variable name triggers a type exception. • Type casting is required when using container objects.

Concise Express much in limited words	**Verbose** Contains more words
Compact	**Less Compact**

Python	**Java**
Uses Indentation for structuring code	**Uses braces for structuring code**

Java Code

```
publicclass HelloWorld
{
publicstaticvoid main (String[] args)
{
System.out.println("Hello, world!");
}
}
```

Python Code

```
print("Hello, world!")
```

Objects:

Definition

- An object is an instance of a class that encapsulates data (attributes) and behavior (methods).
- Everything in Python is an object, including numbers, strings, functions, and classes themselves.

Characteristics:

1. **Attributes**:
   - Objects store data in the form of attributes (variables associated with the object).
   - Example: A `Car` object might have `color` and `speed` as attributes.
2. **Methods**:
   - Functions defined within a class and associated with an object.
   - Example: A `Car` object might have a method `start()`.

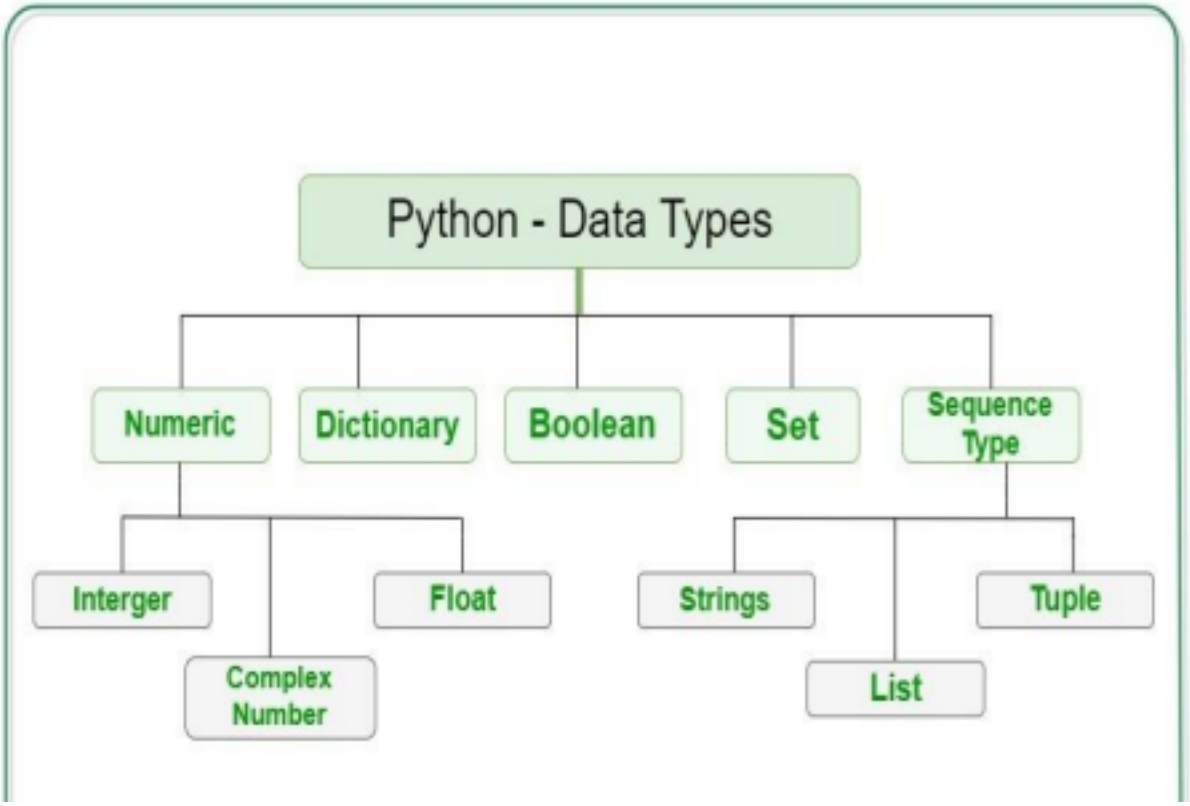
3. **Identity**:
   - Each object has a unique identifier (`id`) that distinguishes it in memory.
   - Use `id(object)` to retrieve the identity.
4. **Type**:
   - The type of an object is its class, which determines its behavior.
   - Use `type(object)` to check the type.
5. **Mutability**:
   - Objects in Python can be mutable (e.g., lists, dictionaries) or immutable (e.g., strings, tuples).

Example

```
class Car:
 def __init__(self, color, speed):
 self.color = color
 self.speed = speed

 def start(self):
 print("Car started")

# Create an object
my_car = Car("red", 120)
print(my_car.color) # Access attribute
my_car.start() # Call method
```



Expressions and Numerical Types:

Expressions

- An expression in Python is a combination of variables, operators, and values that produces a result.
- Examples:
  - Arithmetic: `3 + 5`, `x * y`
  - Logical: `a > b and b < c`
  - Function call: `len(my_list) + 2`

Numerical Types

1. **Integers (`int`)**:
   - Whole numbers, positive or negative, without decimals.
   - Example: `10`, `-5`
2. **Floating-Point Numbers (`float`)**:
   - Numbers with decimal points or in exponential form.
   - Example: `3.14`, `1e-3`
3. **Complex Numbers (`complex`)**:
   - Numbers with a real and imaginary part.
   - Example: `3 + 4j`

4. **Boolean (`bool`)**:
    - A subtype of integers representing `True` (1) or `False` (0).

Variables and Assignments :

Variables

- Variables are used to store data values and act as references to objects in memory. • No need to declare variables explicitly with a type; Python infers the type based on the assigned value.

Variable Naming Rules

1. Names must start with a letter (a-z, A-Z) or an underscore (_).
2. Subsequent characters can include letters, digits (0-9), or underscores.
3. Variable names are case-sensitive (`myVar` and `myvar` are different).
4. Reserved keywords (e.g., `if`, `class`, `def`) cannot be used as variable names.

Assignments

- The assignment operator (=) is used to assign a value to a variable.
- Example:

```
x = 10
```



```
name = "Alice"
```

Multiple Assignments

1. Assign multiple variables in one line:

```
a, b, c = 1, 2, 3
```

2. Assign the same value to multiple variables:

```
x = y = z = 0
```

Dynamic Typing

- Variables can hold values of any type and can be reassigned to different types:

```
x = 10 # Integer
x = "Hello" # String
```

Important Notes

- Variables store references to objects, not the objects themselves.
- Use `id(variable)` to check the memory address of the referenced object.
- Use `type(variable)` to determine the data type of the variable.

Example

```
x = 5 # Integer
y = 3.14 # Float
name = "John" # String
is_valid = True # Boolean
print(x, y, name, is_valid)
```

IDLE :

What is IDLE?

- IDLE (Integrated Development and Learning Environment) is Python's default IDE (Integrated Development Environment).
- It comes bundled with Python installations and provides a user-friendly interface for writing, debugging, and running Python code.

Key Features

1. **Interactive Shell**:
   - A REPL (Read-Eval-Print Loop) environment for executing Python commands interactively.
   - Useful for testing small code snippets.

2. **Code Editor**:
   - Provides a text editor for writing and saving Python scripts with syntax highlighting and auto-indentation.
   - Supports `.py` file creation and execution.
3. **Debugging Tools**:
   - Includes a built-in debugger with features like setting breakpoints and stepping through code.
4. **Output Window**:
   - Displays program outputs, errors, and debugging information in a separate window.
5. **Cross-Platform**:
   - Runs on Windows, macOS, and Linux.

How to Open IDLE

- Installed automatically with Python.
- To open:

- On Windows: Search for "IDLE" in the Start menu.
- On macOS/Linux: Open a terminal and type `idle`.

Advantages

- Simple and lightweight, ideal for beginners.
- No need for additional setup after installing Python.

Limitations

- Lacks advanced features like those found in more robust IDEs (e.g., PyCharm, VS Code).

Example

1. Open IDLE.
2. Write a script in the editor:

```
print("Hello, World!")
```

3. Save the file as `hello.py` and run it using `Run -> Run Module` or `F5`.

Branching Programs :

Definition

- Branching in Python allows a program to make decisions and execute different blocks of code based on conditions.
- Achieved using conditional statements like `if`, `elif`, and `else`.

Key Conditional Statements

1. **`if` Statement**:
   - Executes a block of code if a condition is `True`.
   - Syntax:

```
if condition:
 # Code to execute if condition is True
```

   - Example:

```
age = 18
if age >= 18:
 print("You are an adult.")
```

2. **`if-else` Statement**:

- Provides an alternative block of code if the condition is `False`.
- Syntax:

```
if condition:
 # Code if condition is True
else:
 # Code if condition is False
```

- Example:

```
num = 5
if num % 2 == 0:
 print("Even")
else:
 print("Odd")
```

3. **`if-elif-else` Ladder**:
   - Handles multiple conditions by checking them in sequence.
   - Syntax:

```
if condition1:
 # Code if condition1 is True
elif condition2:
 # Code if condition2 is True
else:
 # Code if none of the above conditions are True o
```

Example:

```
score = 85
if score >= 90:
 print("Grade: A")
elif score >= 75:
 print("Grade: B")
else:
```



```
 print("Grade: C")
```

Logical Operators in Branching

- Combine or modify conditions using:
  - **`and`**: True if both conditions are true.
  - **`or`**: True if at least one condition is true.
  - **`not`**: Reverses the condition's result.
  - Example:

```
if age >= 18 and age < 65:
 print("Eligible for work")
```

Nested Branching

- Conditional statements inside other conditionals for complex decisions. •
Example:

```
num = 10
if num > 0:
 if num % 2 == 0:
 print("Positive even number")
 else:
 print("Positive odd number")
else:
 print("Non-positive number")
```

Branching programs enable dynamic decision-making and are fundamental to writing logical and interactive programs in Python.

Strings and Input :

Strings in Python

- A string is a sequence of characters enclosed in single (`'`), double (`"`), or triple quotes (`'''` or `"""`).
- Strings are immutable, meaning their content cannot be changed after creation.

String Operations

1. **Concatenation**:
   - Combine two or more strings using the `+` operator.
   - Example:

```
greeting = "Hello" + " " + "World"
print(greeting)
```

2. **Repetition**:

   - Repeat a string using the `*` operator.
   - Example:

```
echo = "Hello! " * 3
print(echo)
```

3. **Indexing and Slicing**:
   - Access specific characters or substrings using indexes.
   - Example:

```
text = "Python"
print(text[0]) # First character
print(text[-1]) # Last character
print(text[1:4]) # Substring from index 1 to 3
```

4. **String Methods**:
   - Examples:
     - `lower()`, `upper()`: Convert case.
     - `strip()`: Remove whitespace.
     - `replace()`: Replace parts of a string.
     - `split()`: Split into a list.
     - `join()`: Join elements of a list into a string.

String Formatting

1. **f-strings** (Python 3.6+):

```
name = "Alice"
age = 25
print(f"My name is {name} and I am {age} years old.")
```

2. **`format()` Method**:

```
print("My name is {} and I am {} years old.".format("Alice", 25))
```

Input in Python

- Use the `input()` function to take user input as a string.
- Example:

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

Converting Input

- Input is always treated as a string; convert it to other types when needed.

- Example:

```
age = int(input("Enter your age: "))
print(f"Next year, you will be {age + 1}.")
```

Multiline Strings

- Use triple quotes for multiline strings:

```
message = """This is
a multiline
string."""
print(message)
```

Strings and input handling are essential for interacting with users and manipulating text in Python.

Iteration :

Definition

Iteration refers to the process of repeatedly executing a block of code. In Python, iteration is commonly performed using loops. Python provides two primary types of loops: **for** and **while**.

1. `for` Loop

- The `for` loop iterates over a sequence (such as a list, tuple, string, or range). • Syntax:

```
for variable in sequence:
 # Code to execute for each element in the sequence
```

- Example: Iterating over a list of numbers

```
numbers = [1, 2, 3, 4, 5]
for num in numbers:
 print(num)
```

- **`range()` Function**:
  - Commonly used in `for` loops to generate a sequence of numbers.
  - Syntax: `range(start, stop, step)`
  - Example:

```
for i in range(0, 5):
 print(i)
```

2. `while` Loop

- The `while` loop executes a block of code as long as the condition remains `True`. • Syntax:

```
while condition:
 # Code to execute as long as condition is True
```

- Example: Print numbers from 1 to 5 using a `while` loop

```
count = 1
while count <= 5:
 print(count)
 count += 1
```

3. Iterating Over a Dictionary

- You can iterate over dictionaries to access keys, values, or both.
- Example:

```
my_dict = {'a': 1, 'b': 2, 'c': 3}

# Iterating over keys
for key in my_dict:
 print(key)

# Iterating over values
for value in my_dict.values():
 print(value)

# Iterating over both keys and values
for key, value in my_dict.items():
 print(key, value)
```

4. List Comprehensions (Compact Iteration)

- List comprehensions allow you to create lists based on existing lists or sequences in a compact and readable way.
- Syntax:

```
new_list = [expression for item in iterable if condition]
```

- Example:

```
squares = [x**2 for x in range(5)]
print(squares) # Output: [0, 1, 4, 9, 16]
```

5. `break`, `continue`, and `pass` in Loops

- **`break`**: Exits the loop early.

```
for i in range(10):
```

```
if i == 5:
 break # Exit loop when i equals 5
 print(i)
```

- **`continue`**: Skips the current iteration and moves to the next.

```
for i in range(5):
 if i == 2:
 continue # Skip when i equals 2
 print(i)
```

- **`pass`**: Placeholder; does nothing, useful in empty loops or functions.

```
for i in range(5):
 pass # Does nothing but prevents an error
Example:
for num in range(1, 10): # Loop through numbers from 1 to 9
if num == 5:
 break # Stop the loop entirely when num is 5
 elif num % 2 == 0:
 continue # Skip even numbers
 elif num == 3:
 pass # Do nothing (placeholder), and move to the next
iteration
 print(num)
```

Output

```
1
3
```

Output Explanation:

- `num = 1`: Printed because it doesn't meet any condition.
- `num = 2`: Skipped (`continue` because it's even).
- `num = 3`: `pass` (no action but continues to the next iteration).
- `num = 4`: Skipped (`continue` because it's even).
- `num = 5`: Loop stops (`break`).

Structured Types, Mutability and Higher-order Functions

Tuples :

Definition

- A tuple is an ordered, immutable collection of elements in Python. Once created, the elements of a tuple cannot be modified, which makes tuples useful for storing data that should not be changed.

Creating Tuples

- Tuples are created by placing values inside parentheses `()` and separating them with commas.
- Example:

```
my_tuple = (1, 2, 3, "hello")
```

- A tuple with a single element requires a trailing comma.

```
single_element_tuple = (5,)
```

Accessing Tuple Elements

- Tuples can be accessed using indices, starting from 0.
- Example:

```
my_tuple = (1, 2, 3, "hello")
print(my_tuple[0]) # Output: 1
print(my_tuple[3]) # Output: hello
```

Negative Indexing

- Negative indices access elements from the end of the tuple.
    - `-1` refers to the last element, `-2` to the second last, and so on.
- Example:

```
my_tuple = (1, 2, 3, "hello")
print(my_tuple[-1]) # Output: hello
```

Slicing

- You can slice a tuple to get a subset of its elements.
    - Syntax: `tuple[start:end]` (end is exclusive)
- Example:

```
my_tuple = (1, 2, 3, "hello", 4, 5)
print(my_tuple[1:4]) # Output: (2, 3, 'hello')
```

Immutability

- Tuples are immutable, meaning once a tuple is created, its elements cannot be changed. • Example:

```
my_tuple = (1, 2, 3)
# This will raise an error:
# my_tuple[1] = 4
```

Tuple Operations

1. **Concatenation**: You can concatenate two or more tuples using the `+` operator.

```
tuple1 = (1, 2)
tuple2 = (3, 4)
result = tuple1 + tuple2
print(result) # Output: (1, 2, 3, 4)
```

2. **Repetition**: Repeat a tuple multiple times using the `*` operator.

```
tuple1 = (1, 2)
result = tuple1 * 3
print(result) # Output: (1, 2, 1, 2, 1, 2)
```

3. **Membership Test**: Check if an element is in a tuple using `in` and `not in`.

```
my_tuple = (1, 2, 3)
print(2 in my_tuple) # Output: True
print(4 not in my_tuple) # Output: True
```

4. **Length**: Get the number of elements in a tuple using `len()`.

```
my_tuple = (1, 2, 3)
print(len(my_tuple)) # Output: 3
```

Tuple Packing and Unpacking

1. **Packing**: Assigning multiple values to a single tuple.

```
packed_tuple = 1, 2, 3 # This is equivalent to (1, 2, 3)
```

2. **Unpacking**: Assigning elements of a tuple to multiple variables.

```
my_tuple = (1, 2, 3)
a, b, c = my_tuple
print(a) # Output: 1
print(b) # Output: 2
print(c) # Output: 3
```

Nested Tuples

- Tuples can contain other tuples, which allows for creating complex structures.

```
nested_tuple = (1, (2, 3), (4, 5))
print(nested_tuple[1]) # Output: (2, 3)
print(nested_tuple[1][0]) # Output: 2
```

Purpose for Tuples

- **Data Integrity**: Use tuples when you need to ensure data remains constant and protected from modification.

- **Efficiency**: Tuples are faster than lists because of their immutability.
- **Multiple Return Values**: Functions often return tuples to provide multiple values.

Example of Tuple Usage

```
# Tuple to store coordinates
coordinates = (10.0, 20.5)

# Accessing elements
x, y = coordinates
print(f"x = {x}, y = {y}")
```

Ranges :

Definition

- A range is an immutable sequence of numbers, commonly used in `for` loops for iteration. The `range()` function in Python generates a sequence of numbers, which can be specified using a start, stop, and step value.

Syntax of `range()` Function

```
range(start, stop, step)
```

- **start**: The number where the sequence starts (inclusive). Defaults to `0` if not provided. • **stop**: The number where the sequence ends (exclusive). This argument is required. • **step**: The difference between each number in the sequence. Defaults to `1` if not provided.

Creating a Range

- Example: A simple range from 0 to 4.

```
r = range(5)
print(list(r)) # Output: [0, 1, 2, 3, 4]
```

- Example: A range starting from 2 up to (but not including) 6.

```
r = range(2, 6)
print(list(r)) # Output: [2, 3, 4, 5]
```

- Example: A range with a step of 2.

```
r = range(0, 10, 2)
print(list(r)) # Output: [0, 2, 4, 6, 8]
```

Range with Negative Step

- A negative step allows you to generate a decreasing sequence.


```
r = range(10, 0, -2)
print(list(r)) # Output: [10, 8, 6, 4, 2]
```

Using `range()` in Loops

- `range()` is commonly used in `for` loops to iterate a specific number of times.

```
for i in range(5):
 print(i)
# Output:
# 0
# 1
# 2
# 3
# 4
```

Checking the Type of Range

- The `range()` function returns a special range object, which is not a list but can be converted into a list or iterated over directly.

```
r = range(5)
print(type(r)) # Output: <class 'range'>
```

Converting Range to List or Tuple

- You can convert a `range` object into a list or tuple for easier manipulation.

```
r = range(5)
print(list(r)) # Output: [0, 1, 2, 3, 4]
print(tuple(r)) # Output: (0, 1, 2, 3, 4)
```

Memory Efficiency

- Since a range object is immutable and generates the numbers on-demand (lazy evaluation), it is more memory efficient than creating a list of numbers, especially for large ranges.

Range Examples

1. **Basic Range**:

```
r = range(1, 5)
print(list(r)) # Output: [1, 2, 3, 4]
```

2. **Range with Step**:

```
r = range(0, 10, 3)
print(list(r)) # Output: [0, 3, 6, 9]
```


3. **Reverse Range**:

```
r = range(10, 0, -1)
print(list(r)) # Output: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Common Purpose

- **Iterating over indices**: When you need to iterate over a range of indices, such as when looping through a list or array.

```
fruits = ["apple", "banana", "cherry"]
for i in range(len(fruits)):
 print(i, fruits[i])
```

- **Generating numeric sequences**: For example, generating a series of even or odd numbers.

```
even_numbers = range(0, 20, 2)
odd_numbers = range(1, 20, 2)
```

Important Notes

- `range()` is not inclusive of the `stop` value, meaning it generates values starting from `start` but stops before `stop`.
- A `range()` object is often used with `for` loops because it generates the sequence of numbers dynamically, without storing them in memory all at once.

Lists and Mutability :

Definition of a List

- A **list** is an ordered, mutable (changeable), and indexed collection of elements in Python.

Lists can contain items of any data type (integers, strings, objects, etc.).

- Lists are created using square brackets `[]`.

Creating Lists

- A list is created by placing items inside square brackets, separated by commas.

```
my_list = [1, 2, 3, "apple", 4.5]
```

Accessing List Elements

- List elements are accessed using their index. The index starts from `0`.

```
my_list = [1, 2, 3, "apple"]
print(my_list[0]) # Output: 1
print(my_list[3]) # Output: apple
```

Negative Indexing

- Negative indexing allows you to access list elements starting from the end. `-1` refers to the last element, `-2` refers to the second last element, and so on.

```
my_list = [1, 2, 3, "apple"]
print(my_list[-1]) # Output: apple
```

Modifying Lists

- Lists are **mutable**, meaning you can modify, add, or remove elements. 1.

Changing an element:

```
my_list[0] = 10
print(my_list) # Output: [10, 2, 3, "apple"]
```

2. **Adding elements**:
   - Use `append()` to add an element at the end.

```
my_list.append(5)
print(my_list) # Output: [10, 2, 3, "apple", 5]
```

   - Use `insert()` to insert an element at a specific index.

```
my_list.insert(1, "orange")
  print(my_list) # Output: [10, "orange", 2, 3, "apple", 5]
```

3. **Removing elements**:
   - Use `remove()` to remove an element by value.

```
my_list.remove(3)
  print(my_list) # Output: [10, "orange", 2, "apple", 5]
```

   - Use `pop()` to remove an element at a specific index and return it.

```
my_list.pop(1) # Removes "orange"
print(my_list) # Output: [10, 2, "apple", 5]
```

   - Use `clear()` to remove all elements from the list.

```
my_list.clear()
```



```
print(my_list) # Output: []
```

List Mutability

- The **mutability** of lists allows you to change their content (add, remove, modify elements) after creation.
- Example of mutability:

```
my_list = [1, 2, 3]
my_list[0] = 10 # List content is changed
print(my_list) # Output: [10, 2, 3]
```

Cloning Lists

- You can clone (copy) a list in different ways:

1. **Using the `copy()` method**:

```
original_list = [1, 2, 3]
copied_list = original_list.copy()
copied_list[0] = 10
print(original_list) # Output: [1, 2, 3]
print(copied_list) # Output: [10, 2, 3]
```

2. **Using slicing**:

```
original_list = [1, 2, 3]
copied_list = original_list[:]
copied_list[0] = 10
print(original_list) # Output: [1, 2, 3]
print(copied_list) # Output: [10, 2, 3]
```

3. **Using `list()` constructor**:

```
original_list = [1, 2, 3]
copied_list = list(original_list)
copied_list[0] = 10
print(original_list) # Output: [1, 2, 3]
print(copied_list) # Output: [10, 2, 3]
```

- **Shallow vs. Deep Copy**:
  - A shallow copy (created using `.copy()`, slicing, or `list()`) copies the references to the objects within the list, not the objects themselves.
  - A deep copy (using the `copy` module) copies the objects within the list, creating completely independent copies.

List Comprehension

- **List comprehension** provides a concise way to create lists. It can replace the need for a `for` loop and is often used for filtering or modifying elements.

- Syntax:

```
new_list = [expression for item in iterable if condition]
```

- **Basic Example**:

```
squares = [x**2 for x in range(5)]
print(squares) # Output: [0, 1, 4, 9, 16]
```

- **With Condition**:
  - You can add an `if` condition to filter elements.

```
even_numbers = [x for x in range(10) if x % 2 == 0]
print(even_numbers) # Output: [0, 2, 4, 6, 8]
```

- **Nested List Comprehension**:
  - Creating a list of lists (like a matrix) using nested list comprehensions.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
transposed = [[row[i] for row in matrix] for i in range(3)]
print(transposed) # Output: [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
```

Important List Methods

- `append()` - Adds an item to the end of the list.
- `extend()` - Adds all elements of an iterable to the list.
- `insert()` - Inserts an item at a specific position.
- `remove()` - Removes the first occurrence of an item.
- `pop()` - Removes and returns an item at a given index.

- `sort()` - Sorts the list in place.
- `reverse()` - Reverses the list in place.
- `index()` - Returns the index of the first occurrence of a value.
- `count()` - Returns the number of occurrences of a value.
- `clear()` - Removes all items from the list.

Strings :

Definition

- A **string** in Python is a sequence of characters enclosed in quotes, either single (`'`) or double (`"`). Strings are **immutable**, meaning once a string is created, it cannot be changed.

Creating Strings

- A string can be created using either single quotes (`'`) or double quotes (`"`).


```
string1 = 'Hello, World!'
string2 = "Python Programming"
```

- **Multiline strings** can be created using triple quotes (`'''` or `"""`):

```
multiline_string = '''This is a
multiline string.'''
```

String Indexing and Slicing

- Strings are indexed, starting from `0`. You can access individual characters using the index.

```
my_string = "Hello"
print(my_string[0]) # Output: H
print(my_string[4]) # Output: o
```

- **Negative indexing** allows access from the end of the string:

```
print(my_string[-1]) # Output: o
```

- **Slicing** a string gives a substring using the format `string[start:end]`, where `end` is exclusive.

```
substring = my_string[1:4]
print(substring) # Output: ell
```

String Operations

1. **Concatenation**: You can join two strings using the `+` operator.

```
greeting = "Hello" + " " + "World"
print(greeting) # Output: Hello World
```

2. **Repetition**: You can repeat a string multiple times using the `*` operator.

```
repeated_string = "Hello" * 3
print(repeated_string) # Output: HelloHelloHello
```

3. **Length**: You can get the length of a string using `len()`.

```
my_string = "Python"
print(len(my_string)) # Output: 6
```


4. **Membership Test**: You can check if a substring exists in a string using the `in` or `not in` operators.

```
print("Py" in "Python") # Output: True
print("Java" not in "Python") # Output: True
```

String Methods

1. **`upper()`**: Converts all characters in the string to uppercase.

```
print("hello".upper()) # Output: HELLO
```

2. **`lower()`**: Converts all characters in the string to lowercase.

```
print("HELLO".lower()) # Output: hello
```

3. **`capitalize()`**: Capitalizes the first character of the string.

```
print("hello".capitalize()) # Output: Hello
```

4. **`strip()`**: Removes leading and trailing whitespace characters.

```
print(" Hello ".strip()) # Output: Hello
```

5. **`replace(old, new)`**: Replaces occurrences of the substring `old` with `new`.

```
print("Hello, World!".replace("World", "Python")) # Output: Hello,
Python!
```

6. **`split()`**: Splits the string into a list based on the specified delimiter.

```
sentence = "Hello Python World"
words = sentence.split()
print(words) # Output: ['Hello', 'Python', 'World']
```

7. **`join()`**: Joins the elements of an iterable into a single string, using a specified separator.

```
words = ["Hello", "Python", "World"]
sentence = " ".join(words)
print(sentence) # Output: Hello Python World
```

8. **`find(substring)`**: Returns the index of the first occurrence of a substring or `-1` if not found.

```
print("Hello".find("e")) # Output: 1
print("Hello".find("a")) # Output: -1
```

9. **`count(substring)`**: Returns the number of occurrences of a substring in the string.


```
print("hello hello".count("hello")) # Output: 2
```

10. **`isalpha()`**: Returns `True` if the string contains only alphabetic characters.

```
print("Python".isalpha()) # Output: True
print("Python3".isalpha()) # Output: False
```

11. **`isdigit()`**: Returns `True` if the string contains only digits.

```
print("123".isdigit()) # Output: True
print("123a".isdigit()) # Output: False
```

String Formatting

1. **Old-style formatting (`%`)**:

```
name = "Alice"
age = 30
greeting = "Hello, my name is %s and I am %d years old." % (name, age)
print(greeting) # Output: Hello, my name is Alice and I am 30 years
old.
```

2. **`str.format()` method**:

```
greeting = "Hello, my name is {} and I am {} years old.".format(name,
age)
print(greeting) # Output: Hello, my name is Alice and I am 30 years
old.
```

3. **F-strings (Python 3.6+)**:

```
greeting = f"Hello, my name is {name} and I am {age} years old."
print(greeting) # Output: Hello, my name is Alice and I am 30 years
old.
```

Escape Characters

- You can use escape sequences to represent special characters in a string: ○
  - `\n`: Newline
  - ○ `\t`: Tab
  - ○ `\\`: Backslash
  - ○ `\'`: Single quote
  - ○ `\"`: Double quote

```
print("Hello\nWorld") # Output:
# Hello
# World

print("Hello\tWorld") # Output: Hello World
```


String Immutability

- Strings are **immutable** in Python, meaning their contents cannot be changed after creation.

```
my_string = "Hello"
# This will raise an error:
# my_string[0] = "h"
```

Common Purpose for Strings

- **Storing text data**: Strings are commonly used to store text, such as names, addresses, and descriptions.
- **Processing text**: Many text manipulation tasks such as searching, replacing, and formatting data use strings.
- **User input**: Strings are often used to process user input, either from the keyboard or from files.

Tuples and Lists:

Overview

- **Tuples** and **Lists** are both sequences used to store collections of data in Python, but they have distinct differences in their properties and usage.

Tuples

- A **tuple** is an ordered, immutable collection of elements.
- Once a tuple is created, its elements cannot be modified, added, or removed. •
Tuples are created using parentheses `()`.

Creating a Tuple

```
my_tuple = (1, 2, 3, "apple", 4.5)
```

Accessing Tuple Elements

- Elements in a tuple are accessed via indexing, starting from `0`.

```
print(my_tuple[0]) # Output: 1
print(my_tuple[3]) # Output: apple
```

Tuple Properties

- **Immutability**: You cannot modify, add, or remove elements once the tuple is created.

```
my_tuple[0] = 10 # This will raise a TypeError
```

- **Length**: The length of a tuple can be found using `len()`.


```
print(len(my_tuple)) # Output: 5
```

- **Concatenation and Repetition**: Tuples support concatenation (`+`) and repetition (`*`).

```
new_tuple = my_tuple + (6, 7)
print(new_tuple) # Output: (1, 2, 3, 'apple', 4.5, 6, 7)

repeated_tuple = my_tuple * 2
print(repeated_tuple) # Output: (1, 2, 3, 'apple', 4.5, 1, 2, 3,
'apple', 4.5)
```

Tuple Methods

- **count()**: Counts the occurrences of a specified element in the tuple.

```
my_tuple = (1, 2, 3, 1, 4, 1)
```

```
print(my_tuple.count(1)) # Output: 3
```

- **index()**: Returns the index of the first occurrence of a specified element.

```
print(my_tuple.index(3)) # Output: 2
```

Purpose for Tuples

- **Immutability**: Tuples are often used for data that should not change, such as fixed configuration values or function return values.
- **Performance**: Since tuples are immutable, they are generally faster than lists for iteration and access.
- **Hashable**: Tuples can be used as keys in dictionaries (lists cannot).

Lists

- A **list** is an ordered, mutable collection of elements.
- Lists can be modified after creation by adding, removing, or changing elements. •
Lists are created using square brackets `[]`.

Creating a List

```
my_list = [1, 2, 3, "apple", 4.5]
```

Accessing List Elements

- List elements can be accessed using indexing, similar to tuples.

```
print(my_list[0]) # Output: 1
print(my_list[3]) # Output: apple
```

List Properties

- **Mutability**: Lists can be modified by changing, adding, or removing elements.


```
my_list[0] = 10
print(my_list) # Output: [10, 2, 3, 'apple', 4.5]
```

- **Length**: You can find the length of a list using `len()`.

```
print(len(my_list)) # Output: 5
```

- **Concatenation and Repetition**: Lists support concatenation (`+`) and repetition (`*`).

```
new_list = my_list + [6, 7]
print(new_list) # Output: [10, 2, 3, 'apple', 4.5, 6, 7]

repeated_list = my_list * 2
print(repeated_list) # Output: [10, 2, 3, 'apple', 4.5, 10, 2, 3,
```

```
'apple', 4.5]
```

List Methods

- **append()**: Adds an element to the end of the list.

```
my_list.append(5)
print(my_list) # Output: [10, 2, 3, 'apple', 4.5, 5]
```

- **insert()**: Inserts an element at a specific index.

```
my_list.insert(2, "orange")
print(my_list) # Output: [10, 2, 'orange', 3, 'apple', 4.5, 5] •
```

remove(): Removes the first occurrence of a specified element.

```
my_list.remove("apple")
print(my_list) # Output: [10, 2, 'orange', 3, 4.5, 5]
```

- **pop()**: Removes and returns an element at a specific index.

```
removed_element = my_list.pop(1)
print(removed_element) # Output: 2
print(my_list) # Output: [10, 'orange', 3, 'apple', 4.5, 5] •
```

sort(): Sorts the elements in the list in place.

```
my_list.sort()
print(my_list) # Output: [1, 2, 3, 'apple', 4.5]
```

- **reverse()**: Reverses the list in place.

```
my_list.reverse()
print(my_list) # Output: [4.5, 'apple', 3, 2, 1]
```


Purpose for Lists

- **Mutable Collections**: Lists are ideal for data collections that may change over time, such as inventories, to-do lists, or any sequence that may require modification.
- **Dynamic Data**: Lists are used when the data size is unknown or needs to be altered, such as appending elements or removing items.

Differences Between Tuples and Lists

Feature	Tuples	Lists

Mutability	Immutable	Mutable
Syntax	`()`	`[]`
Performance	Faster (due to immutability)	Slower (due to mutability)
Usage	Fixed collections, as dictionary keys	Dynamic collections
Methods	Fewer methods (e.g., `count()`, `index()`)	Many methods (e.g., `append()`, `insert()`, `remove()`)
Hashable	Yes (can be used as dictionary keys)	No (cannot be used as dictionary keys)
Size	Fixed size	Dynamic size

When to Use Tuples vs. Lists

- **Use Tuples** when:
  - You need an immutable collection of items.
  - You want to ensure data integrity (cannot be accidentally modified).
  - You need to use the collection as a dictionary key.
- **Use Lists** when:
  - You need a collection that can be modified after creation.
  - You need to frequently add, remove, or change elements.
  - You require a dynamic and flexible sequence.

Dictionaries :

Definition

- A **dictionary** in Python is an unordered collection of key-value pairs, where each key is unique. Dictionaries are mutable, meaning they can be modified after creation.

- Dictionaries are defined using curly braces `{}` with key-value pairs separated by colons (`:`).

Creating a Dictionary

You can create a dictionary by enclosing key-value pairs in curly braces:

```
my_dict = {
```

```
 "name": "Alice",
 "age": 30,
 "city": "New York"
}
```

Alternatively, you can use the `dict()` function:

```
my_dict = dict(name="Alice", age=30, city="New York")
```

Accessing Dictionary Elements

- Values in a dictionary are accessed using their **keys**.

```
print(my_dict["name"]) # Output: Alice
print(my_dict["age"]) # Output: 30
```

- You can also use the `get()` method to access a value. It returns `None` if the key doesn't exist (instead of raising an error).

```
print(my_dict.get("city")) # Output: New York
print(my_dict.get("address")) # Output: None
```

Modifying a Dictionary

- **Adding new key-value pairs**: You can add new entries by assigning a value to a new key.

```
my_dict["email"] = "alice@example.com"
print(my_dict)
# Output: {'name': 'Alice', 'age': 30, 'city': 'New York', 'email':
'alice@example.com'}
```

- **Updating values**: You can update the value associated with an existing key.

```
my_dict["age"] = 31
print(my_dict)
# Output: {'name': 'Alice', 'age': 31, 'city': 'New York', 'email':
'alice@example.com'}
```

- **Removing items**: You can remove key-value pairs using `del` or `pop()`. o
  `del`: Removes a key-value pair by specifying the key.


```
del my_dict["city"]
print(my_dict)
# Output: {'name': 'Alice', 'age': 31, 'email':
'alice@example.com'}
```

  - `pop()`: Removes a key-value pair and returns the value associated with the key.

```
email = my_dict.pop("email")
print(email) # Output: alice@example.com
 print(my_dict) # Output: {'name': 'Alice', 'age': 31}
```

Dictionary Methods

1. **`keys()`**: Returns a view object of all the keys in the dictionary.

```
print(my_dict.keys()) # Output: dict_keys(['name', 'age'])
```

2. **`values()`**: Returns a view object of all the values in the dictionary.

```
print(my_dict.values()) # Output: dict_values(['Alice', 31])
```

3. **`items()`**: Returns a view object of all key-value pairs as tuples.

```
print(my_dict.items()) # Output: dict_items([('name', 'Alice'),
 ('age', 31)])
```

4. **`update()`**: Updates the dictionary with key-value pairs from another dictionary or iterable of key-value pairs.

```
new_info = {"city": "Los Angeles", "email": "alice@example.com"}
my_dict.update(new_info)
print(my_dict)
# Output: {'name': 'Alice', 'age': 31, 'city': 'Los Angeles', 'email':
'alice@example.com'}
```

5. **`clear()`**: Removes all key-value pairs from the dictionary.

```
my_dict.clear()
print(my_dict) # Output: {}
```

Dictionary Properties

1. **Mutable**: You can add, remove, or change the contents of a dictionary after it is created. 2. **Unordered**: Dictionaries do not store the items in a particular order (although from Python 3.7 onward, they maintain insertion order, but this should not be relied upon in all cases).
3. **Key Uniqueness**: Each key must be unique in a dictionary. If you try to add a duplicate key, the existing value for that key will be updated.

Nested Dictionaries

- Dictionaries can be nested, meaning a dictionary can contain other dictionaries as values.

```
student_info = {
 "name": "Alice",
 "age": 21,
 "courses": {
 "math": 90,
 "english": 85
 }
}
print(student_info["courses"]["math"]) # Output: 90
```

Dictionary Comprehension

- Similar to list comprehension, you can create dictionaries using a compact syntax.

```
squares = {x: x**2 for x in range(5)}
print(squares)
# Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Purpose for Dictionaries

1. **Storing key-value pairs**: Dictionaries are ideal for storing data where each piece of information has a unique identifier (e.g., storing user details by their username). 2. **Fast lookups**: Since dictionaries are implemented as hash maps, they allow for fast lookups by key, making them efficient for scenarios where quick retrieval is important. 3. **Mapping relationships**: Dictionaries are often used to map relationships between keys and values, such as employee ID to employee details or product ID to product information.